

Dependency Injection (DI) in Spring – Simple Guide

■ What is Dependency Injection?

■ Simple Definition:

Dependency Injection means giving an object its needed things (dependencies) from outside, instead of creating them inside the object.

■ Why Do We Need It?

■ Clean Code

■ Reusability

■ Testability

■■ Loose Coupling

■ Real-Life Example:

Without DI:

```
class Car {  
    Engine engine = new Engine();  
}
```

With DI:

```
class Car {  
    Engine engine;  
    Car(Engine engine) {  
        this.engine = engine;  
    }  
}
```

■ Types of Dependency Injection in Spring

1. Constructor Injection – Pass dependency via constructor
2. Setter Injection – Pass dependency using setter method

■ Example Using Setter Injection in NetBeans

1. Create Maven Project: File → New → Maven → Java App → Name: SpringDIExample
2. Add spring-context dependency in pom.xml
3. Create Interface: MessageService.java
4. Create Class: EmailService.java implements MessageService
5. Create App Class: MyApplication.java with setter method
6. Create XML config: beans.xml in src/main/resources
7. Load context and run in Main.java

■ Other Major Topics in Spring

Spring Core – Base module with DI
Spring Bean – Objects managed by Spring
ApplicationContext – Loads beans from XML/annotations
Spring AOP – Aspect Oriented Programming
Spring Boot – Makes Spring apps easier to build
Spring MVC – Web framework
Spring JDBC – Connect Spring with databases
Spring Annotations – Use @Component, @Autowired, etc.